

This homework has 8 questions, for a total of 100 points and 5 extra-credit points.

Unless otherwise stated, each question requires *clear*, *logically correct*, and *sufficient* justification to convince the reader.

For bonus/extra-credit questions, we will provide very limited guidance in office hours and on Piazza, and we do not guarantee anything about the difficulty of these questions.

We strongly encourage you to typeset your solutions in L^AT_EX.

If you collaborated with someone, you must state their name(s). You must *write your own solution* for all problems and *may not use any other student's write-up*.

(0 pts) 0. **Before you start; before you submit.**

If applicable, state the name(s) and username(s) of your collaborator(s).

Solution: Shikha Nukala, desigirl

(10 pts) 1. **Self assessment.**

Carefully read and understand the posted solutions to the previous homework. Identify one part for which your own solution has the most room for improvement (e.g., has unsound reasoning, doesn't show what was required, could be significantly clearer or better organized, etc.). Copy or screenshot this solution, then in a few sentences, explain what was deficient and how it could be fixed.

(Alternatively, if you think one of your solutions is significantly *better* than the posted one, copy it here and explain why you think it is better.)

If you didn't turn in the previous homework, then (1) state that you didn't turn it in, and (2) pick a problem that you think is particularly challenging from the previous homework, and explain the answer in your own words. You may reference the answer key, but your answer should be in your own words.

Solution: HW9 Q3

Solution:

To show an algorithm to find any Hamiltonian path in G being an undirected graph G , we can show:

if we run algorithm D on an undirected graph G :

- 1) if D returns that there does not exist a path then the algorithm halts on the input and outputs there does not exist a path
- 2) if D returns there does exist a path, then identify any vertex in G called v
- 3) start from v and go through G using an algorithm such as breadth-first search to find a Hamiltonian path in G
- 4) output the path that is found and halt the instance.

From this algorithm layout, we can see that there is a decision on any input, since if there does exist a path then the path is returned by running in polynomial time using a search function while if a path does not exist then the algorithm simply halts anyway. Therefore on all valid inputs the algorithm will output and halt.

We know this is polynomial time complexity because the 3rd step of the algorithm takes polynomial time as the polynomial function of the input size binds the time. The other steps are decision steps and therefore also take polynomial time, therefore the algorithm runs in polynomial time.

For this solution we need to use an efficient algorithm D. Define D as the function that takes in the graph G and constructs G' from G with the Hamiltonian path present in G. Since the size of G does not grow with algorithm D, the algorithm is efficient and runs in some exponential time $O(n^{c+1})$ for a constant c. Constructing the Hamiltonian path at the end of the algorithm is linear, hence the time complexity is $O(n)$ there. Therefore the algorithm D is linear and has been proven. The previous solution did not clearly show the steps taken to solve for the algorithm complexity and needed to outline the algorithm D in a more concise way. Additionally, there is no need to find the Hamiltonian path in the solution. Instead, the path can be constructed at the end of the solution from the input graph G.

2. Job scheduling.

Consider the problem of *fastest-completion job scheduling*: allocating a collection of jobs to servers so as to minimize the time until all the jobs are completed. An input is a number of identical servers n , and the times $t_1, \dots, t_m$ that the m jobs require, i.e., t_i is how long it takes any server to complete job i . The desired output is an allocation of the jobs to the servers having minimum *completion time*, where the completion time of an allocation is the maximum, over all the servers, of the total time of the server's allocated jobs. Each job must be allocated to a single server.

It turns out that this is an NP-complete problem, but there is a polynomial-time (greedy) 2-approximation algorithm:

```
1: function GREEDYALLOCATE( $n, t_1, \dots, t_m$ )
2:   while there is an unallocated job do
3:     allocate an arbitrary unallocated job to a server that would finish its already-allocated jobs earliest
4:   return the computed allocation
```

- (6 pts) (a) Suppose that there are $n = 3$ servers and $m = 7$ jobs, where $t_1 = \dots = t_6 = 1$, and $t_7 = 3$. Determine, with justification, an optimal allocation and its completion time.

Then, consider running GREEDYALLOCATE on this input. Show that, depending on the order in which jobs are allocated by the algorithm, the resulting allocations can have different completion times and approximation factors. Specifically, give two sequences of choices made by the algorithm that result in different completion times, and for each ordering, give its approximation ratio relative to the optimal completion time.

Solution: Since jobs 1-6 take the same time to complete they should be evenly distributed across the three different servers. Therefore the server that is completed the earliest from the three servers after the first 6 jobs should be assigned to the 7th job which takes the longer amount of time with 3 units. Therefore a possible optimal allocation could be

Server 1 = jobs 1, 4, 7

Server 2 = jobs 2, 5

Server 3 = jobs 3, 6

From this solution we can see that the completion takes 4 units of time.

If we run GREEDYALLOCATE on this input then we can get for example these two scenarios:

1:

Server 1 = jobs 1,2,3,4

Server 2 = jobs 5,6,7

In this case the first server takes 4 units of time to run and the second server has 3 extra units of time to run so in total we have 7 units of time taken to complete, which is $7/4$ or 1.75 relative ratio to the optimal time.

2:

Server 1 = jobs 1,2

Server 2 = jobs 3,4

Server 2 = jobs 5,6,7

From this example we can see that the first server and second server take 2 units of time to complete, while the third server takes five units of time to complete because the jobs are consecutive. Therefore this is a $5/4$ or 1.25 relative ratio to the optimal time.

- (6 pts) (b) Prove that when $m \leq n$, GREEDYALLOCATE necessarily returns an optimal allocation.

Solution: When $m \leq n$, we have two scenarios: either $m = n$ or m is less than n . Therefore for both of these separate scenarios we can determine how GREEDYALLOCATE will return an allocation. For $m = n$, each server n is assigned one job (each m). The algorithm allocates jobs by choosing the earliest completion time on each server an assigning a job to that one, so we know that each server will have its minimum completion time running as the algorithm will fill servers that are not in use as it continues. Therefore the algorithm returns the minimum maximum completion time of any server.

For m is less than n , since there are more servers than jobs, it works the same way as $m=n$ where the algorithm allocates jobs to servers not in use or servers that have completed the earliest, which minimizes the maximum completion time of any server by whichever singular job takes the longest to run, therefore the completion time is optimized. Therefore in both cases of $m \leq n$, the completion time returned is necessarily the optimal allocation by the algorithm.

- (6 pts) (c) We have handled the case $m \leq n$, so suppose that $m > n$. Letting OPT be the optimal completion time, prove that $\text{OPT} \geq \max\{t_1, t_2, \dots, t_m\}$, and $\text{OPT} \geq \frac{1}{n} \cdot \sum_{j=1}^m t_j$.

Solution: Proof by contradiction:

If OPT was less than $\max\{t_1, t_2, \dots, t_m\}$, then there would have to be at least one job such that the job t_j would be greater than OPT as given by the definition. However, we know that each server has to complete at least one job by the algorithm and the completion time of any server has to be equal to or greater than the time taken for the longest job entered. So we know the completion time of any server would have to be equal to or greater than t_j which we assume to be a larger job completion time, which would mean that t_j cannot be greater than OPT in any circumstance. Therefore it has to hold that $\text{OPT} \geq \max\{t_1, t_2, \dots, t_m\}$

For $\text{OPT} \geq \frac{1}{n} \cdot \sum_{j=1}^m t_j$, we can prove this with the pigeonhole principle. We know that $\text{OPT} \geq \max\{t_1, t_2, \dots, t_m\}$. Since each server has to complete at least one job, we can apply the pigeonhole principle and say that there exists a server with at least m/n jobs allocated to it as the jobs are distributed by filling every server first. Therefore the optimal allocation of any server must be $\sum_{j=1}^{m/n} t_j$ applying the least time each job takes and the least amount of jobs there can be on that specific server. Therefore we know that the completion time of any server is less than or equal to OPT and greater than or equal to $\sum_{j=1}^{m/n} t_j$ which is the singular server optimal allocation. Therefore if we remove n from the denominator of the summation, we get $\text{OPT} \geq \sum_{j=1}^{m/n} t_j \cdot 1/n \cdot n$ which would become $\text{OPT} \geq 1/n \cdot \sum_{j=1}^m t_j$, which is the statement we have to prove.

- (6 pts) (d) Prove that $\text{ALG} \leq 2 \cdot \text{OPT}$, where ALG is the completion time obtained by GREEDY-LOCATE.

Hint: Consider a server that finishes last (i.e., it finishes at the completion time), and how the jobs were allocated before this server's final job was allocated to it.

Solution: To prove $\text{ALG} \leq 2 \cdot \text{OPT}$, we can start with the completion time of the last server in the allocation being ALG . We can call the last job completed in the algorithm to be t_j and the optimal allocation completion time to be OPT . Therefore we know that t_j comes after all other allocations as it is the last job. Therefore we know that the job j has to be allocated to the server that completed the earliest at that time as the algorithm then could add j to that server. In the algorithm, the completion time t_j is at most the completion time of ALG or the server that finished last as it must be in that server. In the optimal allocation, t_j has to be at most OPT since it must finish by the optimal time. Therefore we can say that $\text{ALG} \geq \text{OPT}$ since OPT is the minimum maximum time and ALG equals OPT at that time or is larger than that time. Therefore if we look at the statement $\text{ALG} \leq 2 \cdot \text{OPT}$, we can see that if ALG is indeed equal to OPT , then $\text{ALG} \leq 2 \cdot \text{OPT}$ should also hold true.

3. Indicator variables and linearity of expectation.

Let $G_{n,p}$ denote a random undirected graph on n vertices, constructed as follows: for each (unordered) pair of distinct vertices $u, v \in V$, include the edge (u, v) in $G_{n,p}$ with probability p , independently of all other random choices.

- (6 pts) (a) Derive the expected number of edges in $G_{n,p}$.

Solution: For each unordered pair of distinct vertices $u, v \in V$, we know that the probability that edge is included in $G_{n,p}$ is p , so the probability it is not included has to be $1 - p$. Therefore we can see that each edge can be a Bernoulli trial with success probability equal to p . Therefore number of edges in $G_{n,p}$ can be N . So for this we can see that the sum of all unordered pair of distinct vertices that make up edges in N is equal to N . We can use truth values and say that the sum adds 1s if there is an edge in N included and adds 0 if there is no edge included. Therefore we know that the expected value of the edges is p and therefore from linearity of expectation we know that we have $\binom{n}{2}$ pairs of distinct vertices in the graph, so the expected number of edges should be $\binom{n}{2} * \text{the probability of each edge or } p$. Therefore the expected number of edges is $\binom{n}{2} \cdot p$.

- (6 pts) (b) Derive the expected degree of any vertex in $G_{n,p}$.

Solution: The expected degree is the number of edges connected to a vertex v . Therefore we can say the degree of vertex x is such that the sum of all edges of two distinct vertices where v is one of those vertices. The edge value is one if the edge is connected to v and 0 if not when adding to the sum. We also know the probability of each edge being included is p , therefore we know that we can use a Bernoulli random variable to represent the success probability of p . From v we know that all other vertices besides v are $n - 1$ in number. So to show all edges in the graph that could be connected to v we have $(n-1) * p$ as the probability, which is the expected degree of any vertex in $G_{n,p}$.

- (6 pts) (c) A *triangle* in a graph is a set of three (distinct) vertices that have an edge between every pair of them. Derive the expected number of triangles in $G_{n,p}$.

Solution: To show the number of triangles in $G_{n,p}$ we have three distinct vertices in this case. If we have vertices x, y, z , we know that the probability they form a triangle is equal to the probability that all three pairs in the three vertices given must have edges between them. Therefore we need to multiply the probability of each edge together three times to get the probability of all three edges being present. Since the probability of one edge is p , the probability of three edges is p^3 . Next we check the sum of indicator to see how we can represent the expected number. If there does exist a triangle between the three, we set the value of those three vertices equal to 1 and add it to the expected sum. If they don't, we set the value of if there is a triangle present to 0. We know that there are $\binom{n}{3}$ ways to pick three vertices that are distinct from the graph. Therefore we take all possible combinations of these triangles and multiply them by the probability that they make up a triangle to get the expected number. Therefore the expected number of triangles is $\binom{n}{3} \cdot p^3$.

- (6 pts) (d) Prove that the number of triangles in $G_{n,p}$ is at least $n^3 p^2$ with probability at most $p/6$.

Solution: We can prove this by setting T to the random number of triangles in the graph. We know that the expected number of triangles is $\binom{n}{3} \cdot p^3$ from the last part. Using this, we apply the Chernoff bound to show that $P(T \geq n^3 p^2) \leq p/6$. Therefore we use the bound for binomial random variables with an expected value and write the equation $P(T \geq 1/2 \cdot \binom{n}{3} \cdot p^3) \leq e^{-\binom{n}{3} p^3 / 8}$. Simplifying by writing out $\binom{n}{3}$ as $n(n-1)(n-2)/6$ on both sides gives us the lower bound for T as $n(n-1)(n-2)p^3 \geq n^3 p^2$. This gives us $p \geq n/(n-1)(n-2)$. We can see that for this inequality $n/2$ is the next largest fraction for the lower bound, so p should be at least $n/2$ for the inequality to be true. Therefore if we go back to the Chernoff bound $P(T \geq 1/2 \cdot \binom{n}{3} \cdot p^3) \leq e^{-\binom{n}{3} p^3 / 8}$ and simplify, we can take the natural log of 6 for the e factor and see that for $p \leq 1$, the log is negative and therefore the inequality holds true that the probability has the upper bound of $p/6$.

4. Randomized max-cut.

In this problem, all graphs are undirected and (as usual) have *no self-loops*, i.e., there is no edge from a vertex to itself. For a cut $S \subseteq V$ in a graph $G = (V, E)$, let $C(S) \subseteq E$ denote the subset of edges “crossing” the cut, i.e., those that have exactly one endpoint in S . The size of the cut is then $|C(S)|$. Consider the following randomized algorithm that outputs a cut in a given graph $G = (V, E)$.

```

1: initialize  $S = \emptyset$ 
2: for all  $v \in V$  do
3:   put  $v$  into  $S$  with probability  $1/2$ , independently of all others
4: return  $S$ 

```

- (7 pts) (a) Define suitable indicator variables and use linearity of expectation to prove that *in expectation*, the above algorithm obtains a $1/2$ -approximation for MAXCUT. That is, the expected size of the output cut is at least half the size of a maximum cut.

Solution: If we define S to be the set of vertices selected and $C(S)$ to be the set of edges crossing the cut from S , then we have indicator variable X_e for each e in E edges and can define the algorithm through the size. Let $|C_{\text{ALG}}|$ be the size of the cut produced by the algorithm, i.e., the number of edges cut by the algorithm. Then:

$$|C_{\text{ALG}}| = \sum_{e \in E} X_e$$

If $|C_{\text{OPT}}|$ is the size of the maximum cut in the graph. By linearity of expectation:

$$E[|C_{\text{ALG}}|] = \sum_{e \in E} E[X_e]$$

The probability that any given edge e is cut by the algorithm (i.e., $E[X_e]$) is the probability that exactly one endpoint of e is included in S , which is $\frac{1}{2}$ (since each

vertex is included in S with probability $\frac{1}{2}$, independently). So, $E[X_e] = \frac{1}{2}$ for all edges e . Therefore:

$$E[|C_{\text{ALG}}|] = \sum_{e \in E} E[X_e] = \frac{1}{2} \sum_{e \in E} 1 = \frac{1}{2}|E|$$

Similarly, $|C_{\text{OPT}}| = \frac{1}{2}|E|$, since in any cut, each edge can be included at most once. So, $E[|C_{\text{ALG}}|] = \frac{1}{2}|E| = \frac{1}{2}|C_{\text{OPT}}|$. So the expected size of the output cut produced by the algorithm is a $1/2$ -approximation for MaxCut.

- (7 pts) (b) Prove that $\Pr[|C(S)| \geq (1 - \varepsilon)|E|/2] \geq \frac{\varepsilon}{1 + \varepsilon}$ for any $\varepsilon > 0$.

Notice that for $\varepsilon = 1/(2|E|)$, this is a lower bound on the probability that the number of edges crossing S is at least $|E|/2$, because the number of crossing edges is an integer.

Solution: To prove this we use linearity of expectation. If C is the random variable for the sum of edges with values of 1 if the edge crosses the cut S , then we have $C = \sum_{e \in E} X_e$ where X_e is the indicator variable for edge e crossing the cut. The probability that a edge crosses the cut S is $1/2$ for each edge and the complement of it. So we have the probability of each edge crossing S as $1/4$. Additionally, the total number of edges crossing S can be written as $|E|/2$ because there are two vertices per edge. Therefore the expected number of edges can be written as $|E|/2 \cdot 1/4$ which would be $|E|/8$. Therefore we have the inequality $P[C \geq (1 - \varepsilon)|E|/2] \leq |E|/8 / (1 - \varepsilon)|E|/2$. This equals $\frac{1}{1 - \varepsilon}$. Therefore the upper bound is less than $(1 - \varepsilon)|E|/2$. Next we can use this to show for $|E|/2$ that $\varepsilon = \frac{1}{2|E|} = (1 - \varepsilon)|E|/2$. Therefore we can write the inequality as $P[C \geq |E|/2] \geq 2|E|/2|E| + 1$, therefore it can be proven that $\Pr[|C(S)| \geq (1 - \varepsilon)|E|/2] \geq \frac{\varepsilon}{1 + \varepsilon}$ for any $\varepsilon > 0$.

- (5 pts) (c) As a stepping stone to the next part, we consider the following intermediate question.

Consider a probability experiment that consists of a sequence of “attempts,” where each attempt succeeds with probability p , independently of all others. We keep making attempts until one succeeds, at which point the experiment terminates.

Let X denote the number of attempts until termination (including the attempt that finally succeeds). Prove that

$$\mathbb{E}[X] = p + (1 - p)(1 + \mathbb{E}[X]) ,$$

and conclude that $\mathbb{E}[X] = 1/p$. (The distribution of X is known as the *geometric distribution* with success probability p .)

Solution: If we have C as the random variable representing the number of attempts until the algorithm stops, then for each attempt the success probability is p . Therefore we know that the first attempt failing has probability $(1 - p)$ which means that the second attempt the expected number is $1 +$ the expected number of attempts until termination. Using this recursion we can see that the expected value of attempts after the first fail is $(1 - p)(1 + \mathbb{E}[C])$. Therefore we can see that the first attempt probability plus the first fail probabilities make the expected value equal to $p + (1 - p)(1 + \mathbb{E}[C])$. To

simplify this, we multiply out the second expression to get $E[C] = 1 + E[C] - pE[C]$ which becomes $E[C] = 1/p$ through factoring again. Therefore we have the expected value as $1/p$ which follows a geometric distribution.

- (6 pts) (d) Suppose we repeatedly run our randomized MAXCUT algorithm until we get a cut of size at least $|E|/2$. Derive an upper bound on the expected number of attempts that are needed.

Solution: Again we can make a random variable C as the number of attempts needed of the algorithm until there is a cut of size at least $|E|/2$. If each attempt of the algorithm is a trial, then the probability of success of getting a cut of size at least $|E|/2$ is p . Then we know from the last part that p is at least $1/2$ and $|E|/2$ is the output. Next we also know that the expected value is $1/p$ from the last part. Therefore if we put in p to the expected value we get $E[C] = 2$. Therefore the upper bound is 2 and the expected number of attempts needed until we get a cut of size at least $|E|/2$ is 2.

- (5 pts) 5. **Quicksort.**

In class, we showed that (randomized) Quicksort has expected running time $O(n \log n)$ on an array of n elements. Although the worst-case running time of the algorithm is $\Omega(n^2)$, we will prove that this happens with small probability.

Let c_1 be the constant such that the expected running time of Quicksort is at most $c_1 \cdot n \log n$ (for all large enough n). Prove that, for any constant $c_2 > 0$, the probability that it takes time at least $c_2 n^2$ time is $O(\frac{\log n}{n})$.

Solution: To prove that the probability of Quicksort taking at least $c_2 n^2$ time is $O(\frac{\log n}{n})$, we can use Markov's inequality. If C is the random variable representing the running time of Quicksort, we know the running time expected of Quicksort is $O(n \log n)$ otherwise written as $c_1 n \log n$ where c_1 is a constant. Therefore if we have another constant c_2 , we should find a probability that C takes at least $c_2 n^2$ time. Therefore if we use the inequality we get $P(C \geq c_2 n^2) \leq E[C]/c_2 n^2$. Therefore if we input $E[C]$ as $c_1 n \log n$ at the very least, we can substitute and get $P(C \geq c_2 n^2) \leq c_1 n \log n / c_2 n^2$ which becomes $P(C \geq c_2 n^2) \leq c_1 \log n / c_2 n$. To show this is $O(\log n / n)$ we know that as n grows larger, the denominator is larger than the numerator, therefore the expression that is on the right side can be expressed as $O(\frac{\log n}{n})$ as the constants do not affect the growth. Therefore we can see that the probability that Quicksort takes at least $c_2 n^2$ time is $O(\frac{\log n}{n})$.

- (12 pts) 6. **Random binary search trees.**

A binary search tree on distinct values is a binary tree where for any node with value x , all the nodes in its left subtree have values less than x , and all nodes in its right subtree have values greater than x . Consider the random process that generates a binary search tree on the integers $\ell, \dots, r$ for given $\ell \leq r$, as follows.

- Choose $x \in \{\ell, \dots, r\}$ uniformly at random and take x as the root. If $\ell = r$, output the tree consisting of just x .
- If $\ell < x$, for x 's left subtree, recursively build a random binary search tree on $\ell, \dots, x - 1$.
- If $x < r$, for x 's right subtree, recursively build a random binary search tree on $x + 1, \dots, r$.

For a random binary search tree on $1, \dots, n$, prove that the expected depth of the node with any particular value is $O(\log n)$.

Hint: Use ideas from the analysis of the Quicksort algorithm from class.

Solution: To prove that the expected depth of the node with any particular value is $O(\log n)$, we can use the Quicksort analysis and recursion tree application. If there is a node in the BST with value c , then we can let C be the random variable representing the depth of the node c . Therefore we need to find the expected value of C or $E[C]$. If n is the total node number in the tree, we can say that there are y nodes with elements less than c and z nodes with number of elements greater than c as that is the structure of a BST. Next if we look at how to build the BST, we know that at each step we choose the root at random from the remaining elements so the expected value for C should be $O(1)$. The left subtree of the node c has y elements and the right subtree has z elements, therefore they are built like Quicksort partitions elements. For the depth of the node we know that if y is smaller than n and z is smaller than n , then the depth of the node c will be $O(\log n)$, otherwise the depth may be larger if either side has node numbers closer to the total number of nodes, that is if c is one of the greatest or least elements. On average then we can say that the number of elements in the left and right subtrees should be closer to $O(N/2)$ if we assume the average to be the even split. Therefore we can conclude the expected depth of the node with any particular value is $O(\log n)$ because at each level of recursion in the tree, the number of elements should decrease by a factor of two and have a height at the end of the tree with $O(\log n)$.

(5 EC pts) 7. **Optional extra-credit question: randomized vertex-cover.**

This question considers randomized algorithms for approximating the vertex-cover problem.

(a) Consider the following probability experiment:

- There is a fixed set of k distinct items.
- The experiment proceeds in a sequence of rounds. In each round, with probability at least $1/2$ (and independently of all other rounds), some arbitrary item that was not previously chosen is chosen. (Otherwise, nothing is chosen in this round.)
- Once every item has been chosen, the experiment ends.

Let R be the random variable denoting the number of rounds until the experiment ends. Prove that $\mathbb{E}[R] \leq 2k$.

Hint: for $j = 1, \dots, k$, let R_j be the random variable denoting the number of rounds after the $(j-1)$ st chosen item and until the j th chosen item (inclusive). What is the relationship between R and the R_j ? What is an upper bound on $\mathbb{E}[R_j]$?

Solution: If R_j be the random variable denoting the number of rounds after the $(j-1)$ st chosen item and until the j th chosen item (inclusive), then R_0 can be 0. Therefore we can say that R or the random variable representing the total number of rounds until all elements are chosen is the $\sum_{j=1}^k R_j$. For each round we can see that the probability of choosing a new item is at least 50 percent or $1/2$. Therefore $p \geq 1/2$. For a geometric random variable we know the probability of success is $1/p$, therefore we can write the expected value as $\mathbb{E}[R_j] \leq 2$. Using linearity of expectation we have $\mathbb{E}[R] = \mathbb{E}[\sum_{j=1}^k R_j] = \sum_{j=1}^k \mathbb{E}[R_j] \leq \sum_{j=1}^k 2 = 2k$. Therefore we have the upper bound of $2k$ from linearity of expectation.

(b) Prove that the following randomized algorithm outputs a 2-approximation, in expectation, for the minimum vertex cover problem.

Hint: let $C^* = \{v_1, \dots, v_k\}$ be some minimum vertex cover in the input graph. Show how a run of the algorithm corresponds to a (complete or partial) run of the experiment from the previous part with the v_i as the items, and use this to bound the expected size of the output C .

```

1: function RANDOMSINGLECOVER( $G = (V, E)$ )
2:    $C = \emptyset$ 
3:   while there is some edge  $e = (u, v)$  that is not covered by  $C$  do
4:     add exactly one of  $u$  or  $v$  to  $C$ , each with probability  $1/2$ 
5:   return  $C$ 

```

Solution: To prove that the following randomized algorithm outputs a 2-approximation, in expectation, for the minimum vertex cover problem, we let $C^* = \{v_1, \dots, v_k\}$ be some minimum vertex cover in the input graph. For each run of the algorithm, in the while loop, each iteration chooses one of the endpoints of the uncovered edge with probability $1/2$. So if we Z be the random variable for the size of the output vertex

cover C , then we can relate the expected size of C to the expected number of rounds in the algorithm since at each round of the algorithm, the algorithm with probability $1/2$ adds one vertex to C which makes the expected size of C half the expected number of rounds until the algorithm stops. Therefore using the last part with $\mathbb{E}[R] = 2k$ with k being the size of the minimum vertex cover, we can write the expected size of the output vertex cover as $\mathbb{E}[Z] = 1/2 * \mathbb{E}[R] \leq 1/2 * 2k = k$. Therefore we can see that the expected size of C is at most twice the size of the minimum vertex cover C^* from the inequality which proves the algorithm is a 2-approximation.